

#Encoder Decoder

```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.reshape(-1, 784) / 255
x_test = x_test.reshape(-1, 784) / 255
```

```
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(64, activation='relu'),
    Dense(128, activation='relu'),
    Dense(784, activation='sigmoid')
])
```

```
model.compile(optimizer='adam', loss='binary_crossentropy')
```

```
model.fit(x_train, x_train, epochs=20, batch_size=256, validation_data=(x_test,
x_test))
```

```
decoded = model.predict(x_test)
```

```
for i in range(10):
    plt.subplot(2, 10, i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.axis('off')

    plt.subplot(2, 10, i+11)
    plt.imshow(decoded[i].reshape(28,28), cmap='gray')
    plt.axis('off')
```

```
plt.show()
```

#OR

```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.reshape(-1, 784)/255; x_test = x_test.reshape(-1, 784)/255
model = Sequential([Dense(64, activation='relu', input_shape=(784,)), Dense(784,
activation='sigmoid')])
model.compile(optimizer='adam', loss='binary_crossentropy')
```

```

model.fit(x_train, x_train, epochs=20, batch_size=256)
decoded = model.predict(x_test)

plt.figure(figsize=(12,3))
for i in range(10):
    plt.subplot(2,10,i+1); plt.imshow(x_test[i].reshape(28,28), cmap='gray');
plt.axis('off')
    plt.subplot(2,10,i+11); plt.imshow(decoded[i].reshape(28,28), cmap='gray');
plt.axis('off')
plt.tight_layout(); plt.show()
=====
=====
#LSTM

import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

df = pd.read_csv("VegetablePrice.csv")
# Use only Tomato Big(Nepali) - sort by date
data = df[df['Commodity'] == "Tomato Big(Nepali)"].sort_values('Date')
avg = data['Average'].values.reshape(-1, 1)

# Normalize
scaler = MinMaxScaler()
avg_scaled = scaler.fit_transform(avg)

# Create sequences
seq_len = 10
X, y = [], []
for i in range(len(avg_scaled)-seq_len):
    X.append(avg_scaled[i:i+seq_len])
    y.append(avg_scaled[i+seq_len])
X, y = np.array(X), np.array(y)

# Build and train model
model = Sequential([
    LSTM(50, input_shape=(seq_len, 1)),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=10, batch_size=32)

# Prediction and inverse-transform
pred = model.predict(X)
pred = scaler.inverse_transform(pred)
print(pred)

```

#OR

```
import pandas as pd, numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("VegetablePrice.csv")
avg = df[df['Commodity']=="Tomato
Big(Nepali)"].sort_values('Date')['Average'].values.reshape(-1,1)
scaler = MinMaxScaler(); avg_scaled = scaler.fit_transform(avg)
seq_len = 10
X = np.array([avg_scaled[i:i+seq_len] for i in range(len(avg_scaled)-seq_len)])
y = avg_scaled[seq_len:]
```

```
model = Sequential([LSTM(32, input_shape=(seq_len,1)), Dense(1)])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=10, batch_size=32)
```

```
pred = scaler.inverse_transform(model.predict(X))
```

```
=====
==
```

#Pretrained

```
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
```

```
base = ResNet50(weights='imagenet', include_top=False, input_shape=(224,224,3))
for l in base.layers: l.trainable = False
```

```
model = Sequential([
    base,
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

#if asked to train the model ('Optional')

```
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
```

```
(x, y), _ = fashion_mnist.load_data()
```

```
x = tf.image.resize(tf.expand_dims(x[:128], -1), (224,224)); x =  
tf.image.grayscale_to_rgb(x) / 255.  
model.fit(x, to_categorical(y[:128], 10), epochs=2)
```